

УДК 004.421.6

ФРЕЙМВОРК ДЛЯ РАЗРАБОТКИ ИНТЕЛЛЕКТУАЛЬНЫХ ОБУЧАЮЩИХ СИСТЕМ НА ОСНОВЕ МОДЕЛЕЙ ПРЕДМЕТНЫХ ОБЛАСТЕЙ В ВИДЕ ДЕРЕВЬЕВ РЕШЕНИЙ¹

А.И. Крыгин (*andrey_krygin@outlook.com*)

М.Р. Гумеров (*max_person@mail.ru*)

Н.А. Москаленко (*nekit_vlg@mail.ru*)

О.А. Сычев (*oasychev@gmail.com*)

Волгоградский Государственный Технический Университет,
Волгоград

Формальные модели предметных областей позволяют строить интеллектуальные обучающие системы основанные на символьном интеллекте с надежным определением ошибок обучаемых, генерацией объясняющей обратной связи, наводящих вопросов и другими предсказуемыми свойствами. Однако разработка таких моделей для современных машин вывода является трудоемкой. В работе описывается способ представления решения и проверки задач предметных областей в виде деревьев решений, а также прототип фреймворка для разработки интеллектуальных обучающих систем на его основе. По результатам опытной разработки четырех ядер интеллектуальных обучающих систем, фреймворк показал умеренную производительность и значительное сокращение сроков разработки.

Ключевые слова: деревья решений, интеллектуальные обучающие системы, модели предметных областей, фреймворк.

Введение

Интеллектуальная обучающая система (ИОС, Intelligent Tutoring System) – это программное обеспечение, созданное для имитации навыков, поведения и указаний преподавателя. ИОС разрабатываются с использованием технологий ИИ, пытаясь приблизиться к техникам обучения, которые, при их выполнении человеком, можно было бы назвать “интеллектуальными”. [Nwana, 1990] Преимуществами ИОС по сравнению с преподавателем-человеком являются возможность охвата

¹ Исследование выполнено при финансовой поддержке ВолгГТУ в рамках научного проекта № 60/438-22.

большого количества учащихся, а также ведение проработки темы до овладения ею (training until mastery) без ограничений по времени.

Частыми функциями ИОС являются способность анализировать сложные ответы учащегося и адаптироваться к ним в процессе работы, возможность определить, где и почему учащийся допустил ошибку, и дать необходимые подсказки, чтобы помочь ему в понимании материала.

В традиционной форме ИОС описываются как оперирующие информацией трех ключевых видов: информацией о предметной области, учащемся, и способах или стратегиях обучения - именно эти данные обеспечивают “интеллектуальность” системы. В классических архитектурах ИОС выделяют 3 соответствующих информационных модели (модель предметной области, модель учащегося, и педагогическая модель), а также модуль, ответственный за пользовательский интерфейс и его особенности. [Nwana, 1990], [Alkhatlan et al., 2018]

В силу такого разделения, большинство независимых ИОС включают в себя специфическую реализацию трех описанных моделей. Большинство современных ИОС преуспевают только в узком кругу задач, и неразрывно связаны с представляемой ими предметной областью. [Soofi et al., 2019], [Mousavinasab et al., 2018]

Для создания универсальных ИОС, основанных на моделях предметных областей, необходимо формальное описание модели предметной области; оно используется как входные данные для обучающей системы, делая саму систему не зависящей от специфики изучаемых проблем. Небольшое количество ИОС реализует данный подход. [Freedman et al., 2000], [Singh et al., 2022] При этом по-прежнему может стоять необходимость разработки отдельного пользовательского интерфейса для каждой предметной области, однако наибольшие трудозатраты всегда связаны с моделированием предметных областей. Их минимизация наиболее актуальна для широкого распространения ИОС.

[Turner et al., 2005] реализовывали инструмент ASSISTment Builder для создания быстрых “псевдо-тренажеров”, использующих когнитивную модель, основанную на графе состояний, настроенном для конкретной задачи ее автором. Однако следует отметить, что ИОС, разработанные с использованием этого инструмента, имеют ограниченную область применения, поскольку предназначены для решения одной учебной задачи каждая. Процесс создания ИОС приходится повторять для каждой задачи.

[O’Rourke et al., 2015] предлагают формализацию процессов решения класса проблем, в виде кода на независимом от предметной области языке, называемом языком мыслительных процессов (Thought Process Language, TPL). По мере выполнения TPL кода система регистрирует факты выполнения определенных операторов языка, которые затем могут быть отображены с помощью “зацепок” интерфейса, специфичных для

предметной области. Такое представление мыслительного процесса в виде абстрактного кода, хотя и является перспективным подходом, ограничено представлением TPL – представление мыслительного процесса обучаемого в ИОС базирующееся на прямом соответствии программному языку заставляет его мыслить в терминах этого языка (присвоениях, развилках, циклах), что не является естественным для человеческого мышления (основанного на задавании вопросов, отбору по критериям и принятии решений). Кроме того, TPL не описывает поиск семантических ошибок и генерацию их объяснений; он позволяет только подсказывать правильные решения.

[Singh et al., 2022] описывают высокоуровневую модель предметной области, состоящую из тем, зависящих друг от друга (если для понимания одной темы необходимо первоначально ознакомиться с другой) и объединенные в курсы, также зависящие от тем и друг от друга. Данная схема используется в совокупности с классификацией запросов учащихся на соответствие темам, и использование экспертов предметной области для реализации новых тем обеспечивает системе расширяемость. Эта высокоуровневая модель не моделирует мышление учащихся при освоении конкретных тем и может использовать только при планировании занятий.

Одной из основных проблем использования ИОС в общем является трудоемкость их разработки особенно при использовании стандартных методов хранения баз знаний и машин вывода на их основе. Без использования заменяемой модели предметной области ИОС должна обладать всеми необходимыми экспертными знаниями, и оценки времени разработки варьируются от 200 до 300 часов разработки на 1 час обучающего контента [Aleven et al., 2006]. Хотя использование формальной модели предметной области и предоставляет возможность повторного использования и быстрого изменения необходимой модели, ее построение остается трудозатратным процессом, что обусловлено также тем, что подобные модели должны быть описаны в более общих терминах, нежели интегрированные, специфичные модели. [Martin et al., 2003], [Mitrovic et al., 2007] В связи с этим важным является облегчение труда разработчиков обучающих моделей предметных областей и повышение его производительности [O'Rourke et al., 2015], [Aleven et al., 2006], [Mitrovic et al., 2007].

В данной работе мы рассматриваем новый подход к представлению формальных моделей предметных областей, а также фреймворк, использующий данный подход для облегчения построения ИОС, независимых от предметной области.

1. Деревья решений и задачи в интеллектуальных обучающих системах, решаемые с их помощью

При определении формата описания предметной области для ИОС необходимо учитывать стоящие перед системой задачи, т.к. ее описание должно включать в себя всю необходимую информацию для их решения. В данной работе мы основываемся на моделировании предметных областей в виде деревьев решений. В общем случае такие деревья моделируют процессы рассуждений предметной области, что открывает возможности решений различных педагогических задач. Деревья могут отвечать на разные вопросы, например: “Какой следующий шаг является правильным для решения задачи?” или “Корректен ли данный шаг в решении задачи и если нет, то почему?”

Деревья решений описываются узлами (шагами рассуждения) и переходами между ними:

- Узлы вопросов - принять решение в зависимости от некоторого условия (ответа на некоторый вопрос);
- Узлы ветвления - принять решение в зависимости от результатов нескольких подзадач (рассуждений более низкого уровня);
- Узлы результатов означают получение ответа на (под)задачу;
- Узлы действий - определить ключевые объекты задачи для использования в дальнейших рассуждениях;
- Циклические узлы - выполнить некоторые рассуждения для набора ключевых объектов.

Для связи с формальным условием задачи и ответов обучаемого, эти деревья сопровождаются словарями понятий предметной области:

- Классы определяют категории или типы объектов в предметной области. Они поддерживают наследование позволяя определять общие характеристики для группы понятий;
- Свойства описывают атрибуты или характеристики объектов некоторого класса;
- Отношения задают связи и взаимоотношения между объектами указанных классов.

Процесс рассуждения в узлах дерева решений представляет собой вычисление выражений логики первого порядка над базой знаний о конкретной задаче. Структура связей между узлами задает общий процесс рассуждений.

Деревья решений могут моделировать две категории процессов:

- Процесс решения задач - рассуждения, которые необходимы непосредственно для нахождения (создания или определения) правильного ответа на конкретную задачу. Деревья этой категории назовем деревьями вычислений.

- Процесс проверки ответов на задачи - рассуждения, необходимые для проверки частичного ответа обучаемого на корректность. Такие деревья назовем деревьями проверок.

Эти категории деревьев предоставляют различные возможности, разрабатываемой ИОС. Деревья вычислений дают возможность:

- “Прямого” решения задач путем автоматического выполнения рассуждений, предписанных деревом вычислений. ИОС может найти корректный ответ на любую из своих задач, и сравнить ответ учащегося с корректным. Это значительно облегчает как ввод задач учителем, так и их автоматическую генерацию.

- Генерации интеллектуальных подсказок, объясняющих один правильный шаг в решении задачи. Задействованные узлы дерева решений позволяют генерировать объяснения, почему данный шаг в данных обстоятельствах правильный.

Деревья проверок позволяют:

- Определять семантические ошибки обучаемого - узлы результаты в дереве проверок прямо соответствуют ограничениям (constraints) предметной области, которые были нарушены в данном ответе, что позволяет организовать обучающую обратную связь..

- Генерировать наводящие вопросы для определения конкретных ошибок в мышлении пользователя. С их помощью можно установить ошибки в знании фактов, правил или порядка их применения.

Таким образом, моделирование предметных областей с помощью деревьев решений обеспечивает хорошую основу для построения ИОС, учитывающих процессы рассуждений учащихся.

2. Разработанный Фреймворк

Разработанный фреймворк состоит из трех частей: графического конструктора деревьев решений предметной области, машины вывода, и генератора наводящих вопросов.

Графический конструктор был реализован на базе редактора диаграмм Draw.io [diagrams.net, 2021]. Данный редактор имеет открытый исходный код и возможность разработки плагинов. В разработанном авторами плагине была добавлена возможность создания и редактирования узлов деревьев решений, описанных ранее, и элементов, содержащих словари используемых классов, свойств и отношений. Узлы содержат дополнительную информацию в зависимости от их вида. Связи между узлами представлены стрелками, которые хранят значения возможного результата выполнения узла.

Формальным описанием действий с базой знаний, необходимых для выполнения узлов, служат выражения рассуждений. Для записи этих

выражений был разработан специальный язык логики первого порядка для удобной обработки графов знаний (например, представленных в формате RDF). Этот язык включает в себя логические операции и кванторы; он позволяет выполнять операции проверки, получения и изменения информации в графовой базе знаний, а также оперировать переменными как в рамках одного выражения, так и в рамках всего дерева решений. Значениями данных переменных являются объекты базы знаний задачи. Свойства объектов могут хранить данные различных типов: целые и дробные числа (ограниченные в некоторых пределах или неограниченные), булевы значения, строки и заданные автором перечисления.

На рис. 1 приведены примеры выражений на разработанном языке.

<pre>find C { \$C is operand and \$C->isInside(var:X) and \$C.state == state::evaluated }</pre>	<pre>forall X_t [var:X->has(\$X_t)] { \$X_t->leftOf(var:Y2) and \$X_t->rightOf(var:Y1) }</pre>
(a)	(б)

Рис. 1. Пример выражения-действия (а) и выражения-утверждения (б).

Выражение, изображенное на рис. 1, а, осуществляет поиск объекта *C*, который имеет класс *operand*, находится внутри объекта, сохраненного в переменную *X* (связан отношением *isInside* с этим объектом) и имеет свойство *state* со значением *unused*. Выражение, изображенное на рис. 1, б, проверяет логическую верность утверждения, что для всех *X_t*, связанных с переменной *X* отношением *has* верно утверждение, что *X_t* находится справа от переменной *Y1* (связан отношением *leftOf*) и слева (отношение *rightOf*) от *Y2*.

Выражения для узлов могут задаваться как с помощью текстовой записи, так и визуально посредством блоков из библиотеки Blockly [Pasternak et al., 2017], представляющих отдельные операторы языка. Блоковое представление позволяет уменьшить время, требуемое для изучения синтаксиса текстовой записи разработанного языка и исключить синтаксические ошибки.

Созданное в данном конструкторе дерево решений является основой для решения всех описанных в предыдущем разделе задач. Для этого необходимо выполнять как отдельные узлы, так и ветви дерева в целом; этим занимается машина вывода. Простейшим вариантом ее реализации является интерпретатор дерева решений, который был реализован авторами для оценки перспектив фреймворка.

Более эффективной реализацией было бы преобразование всего дерева решений или отдельных узлов в продукционные правила, используемые традиционными машинами логического вывода, в частности Generic Rule Reasoner из фреймворка Apache Jena [Jena, 2023], показывающий лучшие результаты по скорости работы на больших графах знаний среди сравнимых машин вывода [Sychev et al., 2021]. Но такое преобразование является сложной задачей. В Apache Jena ограничены способы конструирования правил. Основными проблемами являются отсутствие дизъюнкции, отсутствие отрицания над группой утверждений, отсутствие возможности задания приоритета правилам, а также сложности при организации итеративных действий. Например, при преобразовании выражения, изображенного на рис. 1, (а), в продукционные правила Apache Jena, получается правило, изображенное на рис. 2.

```
[
  (?C rdf:type ?var2...)
  isReachable(?var2...,rdfs:subClassOf,poas:element,"true"^^xsd:boolean)
  noValue(?C,poas:state,poas:unevaluated)
  (?var3... poas:var... "X"^^xsd:string)
  (?C poas:isInside ?var3...)
  (?C poas:state ?var5...)
  equal(?var5...,poas:evaluated)
  makeSkolem(?var1...)
  ->
  (?var1... poas:predicate1... ?C)
]
```

Рис 2. Пример правила Jena для вычисления выражения, изображенного на рис. 1, а.

Преобразование выражения, изображенного на рис. 1, б, потребует несколько наборов правил, объем которых в 4.3 раза превышает объем правила на рис. 2. Составление таких правил требует от разработчика четкого понимания механизма работы продукционных правил Jena и умения организовывать работу нескольких, независимо выполняющихся правил. Поэтому генерация правил Jena из деревьев решений предпочтительна их ручной разработке.

Формальное представление процесса решения задач в виде дерева решений также открывает возможности для решения более сложных задач. Например, с их помощью можно генерировать наводящие вопросы. [Sychev et al., 2023] Структура дерева проверок для конкретной задачи служит входными данными для создания конечного автомата вопросов, позволяющего в каждой конкретной ситуации ошибки учащегося эффективно определить, в чем именно была ошибка рассуждений

учащегося, и объяснить ему ее причину. Возможность ведения диалога с помощью наводящих вопросов приближает интеллектуальные тренажеры к взаимодействию с преподавателем-человеком.

3. Апробация

С использованием разработанного фреймворка были разработаны четыре ядра ИОС для обучения следующим интеллектуальным навыкам: определение порядка вычисления операций в выражении на языках программирования; определения порядка прилагательных в словосочетании на английском языке; составление выражений доступа к данным в языках программирования; разрешение имен в программных выражениях. Последние три из них были разработаны авторами не входящими в коллектив разработчиков фреймворка.

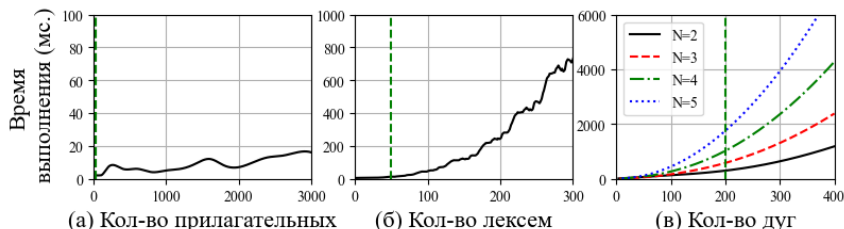


Рис. 3. Графики зависимости времени выполнения рассуждений, описанных деревом решений, от показателя сложности задачи

На первых трех предметных областях были проведены замеры скорости проверки ответа студента; ее результаты для трех предметных областей в зависимости от сложности задач представлены на рисунке 3.

Вертикальная черта на каждом из графиков обозначает максимальную сложность задач, которую эксперты областей сочли подходящей для обучения. Для навыка “определение порядка прилагательных” реалистичный размер словосочетания составляет не более 20 слов. Для определения порядка вычисления выражения объем задачи измерялся в количестве лексем в выражении (до 50 лексем). Для навыка “составление выражений доступа к данным” сложность задачи зависит от количества дуг в графе возможных путей к данным; т.к. производительность обучающей системы в этой задаче зависит от структуры графа путей, были проведены замеры доступа к элементам массивов увеличивающейся размерности из N элементов по каждому измерению (см. рис. 3 в).

Из графика можно видеть, что на понятных человеку объемах данных в первых двух областях (рис. 3. а, б) интерпретатор тратит на рассуждения приемлемое количество времени, однако в области выражений доступа (рис. 3. в), на решение реальной по объему задачи, может уходить до 2-х секунд. Соответственно, интерпретатор в целом позволяет разрабатывать эффективные тренажеры с пошаговой оценкой данных, но в некоторых предметных областях он не является масштабируемыми для более сложных задач. Поэтому интерпретатор возможность автоматического преобразования логических выражений в продукционные правила современных машин вывода является перспективным направлением дальнейших исследований.

Для определения порядка операторов в выражении было разработано дерево проверок, состоящее из 62 узла и 41 логического выражения. Результирующая ИОС может использовать 74 наводящих вопроса, каждый из которых распознает одну уникальную ошибку в мышлении учащегося. Разработка интеллектуального ядра тренажера с помощью дерева решений и фреймворка заняла около 20 человеко-часов. Ручное написание продукционных правил Apache Jena Generic Rule Reasoner для этой задачи включало составление 230 правил и потребовало около 150 человеко-часов. Это показывает что фреймворк позволяет значительно снизить трудозатраты на разработку моделей предметных областей.

Оценки времени, затраченного на разработку модели предметной области тремя различными разработчиками, не являвшимися авторами фреймворка представлены в таблице 1.

Табл. 1.

Предметная область	Кол-во узлов в дереве решений	Кол-во логических выражений	Примерное время разработки, ч.
Порядок прилагательных	47	21	10
Выражения доступа	25	11	20
Разрешение имен	136	72	50

4. Заключение

Мы предложили перспективный подход к представлению предметных областей ИОС на основе деревьев решений, позволяющий решить серию

задач, необходимых при разработке интеллектуальных обучающих систем при существенном снижении трудоемкости разработки. С использованием данного подхода был разработан фреймворк для создания ИОС на базе деревьев решений.

Данный фреймворк решает задачи выполнения необходимых рассуждений на базе знаний, а также генерации наводящих вопросов в разработанной ИОС. Однако используемый на данный момент в качестве машины вывода интерпретатор не масштабируем для работы с большими базами знаний. Преобразование разработанных логических выражений в продукционные правила Apache Jena или других машин вывода, таких как SWI-Prolog [Wielemaker et al., 2011], Answer Set Programming (ASP) [Gebser et al., 2018], Pellet [Sirin et al., 2007] и SPARQL [SPARQL, 2008], является приоритетным направлением дальнейших исследований.

Другим направлением дальнейших исследований является расширение функциональных возможностей фреймворка - например, генерация интеллектуальных подсказок, преобразование деревьев проверок в деревья вычислений и наоборот, а также определение списка задач, которые необходимо проверить для оценки усвоения предметной области.

Список литературы

- [Nwana, 1990] Nwana H. Intelligent tutoring systems: an overview // *Artificial Intelligence Review*. 1990. № 4(4).
- [Alkhatlan et al., 2018] Alkhatlan A. and Kalita J.K. Intelligent Tutoring Systems: A Comprehensive Historical Survey with Recent Developments. // *International Journal of Computer Applications*. 2018. № 181(43).
- [Freedman et al., 2000] Freedman R., Ali S.S. and McRoy S. Links: what is an intelligent tutoring system? // *Intelligence*. 2000. № 11(3).
- [Singh et al., 2022] Singh S. and Singh V. An Architecture of Domain Independent and Extensible Intelligent Tutoring System based on Concept Dependencies and Subject Paths // *International Journal of Advanced Computer Science and Applications*. 2022. № 13(5).
- [Turner et al., 2005] Turner T.E., Macasek M.A., Nuzzo-Jones G., Heffernan N.T. and Koedinger K. The Assistment Builder: A Rapid Development Tool for ITS // *Proceedings of the 12th Artificial Intelligence In Education*. 2005.
- [O'Rourke et al., 2015] O'Rourke E., Andersen E., Gulwani S. and Popović Z.A. Framework for Automatically Generating Interactive Instructional Scaffolding // *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*. 2015.
- [Aleven et al., 2006] Aleven V., McLaren B.M., Sewall J. and Koedinger K.R. The Cognitive Tutor Authoring Tools (CTAT): Preliminary Evaluation of Efficiency Gains // *In the Proceedings of the 8th International Conference on Intelligent Tutoring Systems*. 2006.

- [Martin et al., 2003] Martin B. and Mitrovic A. ITS Domain Modeling: Art or Science? // In the Proceedings of the 11th International Conference Artificial Intelligence in Education. 2003.
- [Mitrovic et al., 2007] Mitrovic A., Brent M. and Pramuditha S. Intelligent Tutors for All: The Constraint-Based Approach // Intelligent Systems, IEEE. 2007. № 22(4).
- [Sychev et al., 2021] Sychev O.A., Anikin A. and Denisov M. Inference Engines Performance in Reasoning Tasks for Intelligent Tutoring Systems // International Conference on Computational Science and Its Applications. 2021.
- [Soofi et al., 2019] Soofi A.A. and Ahmed M.U. A Systematic Review of Domains, Techniques, Delivery Modes and Validation Methods for Intelligent Tutoring Systems // International Journal of Advanced Computer Science and Applications. 2019. № 10(3).
- [Mousavinasab et al., 2018] Mousavinasab E., Zarifasanaiey N., Niakan Kalhori S.R., Rakhshan M., Keikha L. and Ghazi Saeedi M. Intelligent tutoring systems: a systematic review of characteristics, applications, and evaluation methods // Interactive Learning Environments. 2018. № 29(1).
- [diagrams.net, 2021] diagrams.net, previously draw.io, is an online diagramming web site that delivers the source in this project. [Электронный ресурс]. URL: <https://app.diagrams.net/> (Дата обращения: 27.05.2023).
- [Pasternak et al., 2017] Pasternak E., Fenichel R. and Marshall A.N. Tips for creating a block language with blockly // IEEE Blocks and Beyond Workshop. 2017.
- [Sychev et al., 2023] Sychev O. and Gumerov M. Generating Pedagogical Questions to Help Students Learn // International Conference on Intelligent Tutoring Systems. 2023.
- [Wielemaker et al., 2011] Wielemaker J., Schrijvers T., Triska M. and Lager T. SWI-Prolog // Theory and Practice of Logic Programming. 2011. № 12(1-2).
- [Gebser et al., 2018] Gebser M., Kaminski R., Kaufmann B. and Schaub T. Multi-shot ASP solving with clingo // Theory and Practice of Logic Programming. 2018. № 19(1)
- [Sirin et al., 2007] Sirin E., Parsia B., Grau B.C., Kalyanpur A. and Katz Y. Pellet: A practical OWL-DL reasoner // Web Semantics: Science, Services and Agents on the World Wide Web. 2007. № 5(2).
- [Jena, 2023] Jena - a free and open source Java framework for building Semantic Web and Linked Data applications. [Электронный ресурс]. URL: <https://jena.apache.org> (Дата обращения: 29.05.2023).
- [SPARQL, 2008] SPARQL Update. A language for updating RDF graphs. W3C Member Submission 15 July 2008. [Электронный ресурс]. URL: <https://www.w3.org/Submission/SPARQL-Update/> (Дата обращения: 29.05.2023).

A FRAMEWORK FOR DEVELOPING INTELLIGENT TUTORING SYSTEMS BASED ON DOMAIN MODELS IN THE FORM OF DECISION TREES

A.I. Krygin (andrey_krygin@outlook.com)
M.R. Gumerov (max_person@mail.ru)

N.A. Moskalenko (nekit_vlg@mail.ru)

O.A. Sychev (oasychev@gmail.com)

Volgograd State Technical University, Volgograd

Formal domain models allow the development of intelligent tutoring systems based on symbolic AI capable of reliable identification of learners' errors, generation of explanatory feedback, pedagogical questions, and other predictable features. However, the development of such models for modern inference machines is time-consuming and labor-intensive. The paper describes a method for representing processes of solving and verifying problems of subject domains in the form of decision trees, as well as a prototype of a framework for developing intelligent learning systems based on that method. According to the results of the development of four intelligent tutoring systems' cores, the framework showed moderate performance and a significant decrease of development time.

Keywords: decision trees, intelligent tutoring systems, domain models, framework.